Lambda World speaker script

4,044 spoken words. Planning pace: 125 words/minute, plus 04:16 of pauses and visual holds. Speech and holds: 36:37; with 02:00 for questions: 38:37. This is an estimate, not an aloud rehearsal.

Speak the prose; square-bracketed stage cues in exports are not spoken. Long numerical traces have one continuous passage plus a final-state passage. No Clojure or OR prerequisite; explain notation at first use. Timing is a calculation, not an aloud rehearsal.

01 Rebuilding Operations Research in Higher-Order Functions

00:00–00:12 · 19 words · 3s visual holds

Phase 1

> Let the title settle.

Hello, I'm Jiri. We'll build three operations research examples with higher-order functions, then examine a proof about an update.

02 About me

00:12–00:42 · 55 words · 3s visual holds

Phase 1

> Introduce your background briefly; the interviews line links to the Flexiana channel.

I started programming thirty-four years ago, and I've been using Clojure for fourteen years. My education is in economics. Thirteen years ago, I started the Barcelona Meetup Group. I also interview many Clojure open-source authors on Flexiana's YouTube channel. This talk brings together my interests in functional programming, mathematical models, and decisions we can explain.

03 Why Clojure?

00:42–01:09 · 51 words · 3s visual holds

Phase 1

> Move down the four reasons.

I chose Clojure for small functions, immutable data, and REPL-oriented programming. Keeping old states makes experiments easy to compare. Code is also data, which we'll use later when we get to proofs. You don't need Clojure experience today. I'll introduce its notation where it matters; the functional ideas should be familiar.

04 Operations research

01:09–01:47 · 72 words · 3s visual holds

Phase 1

> Leave the concrete examples for later.

Operations research uses models to compare choices under objectives and constraints. Optimization searches for a plan. Decision-making under uncertainty compares possible outcomes.

I also include validation: checking whether a system respects its specification. Reachability usually belongs to verification; it matters here because a profitable decision system can still violate a requirement. These are different questions. We'll use a different kind of state for each, but the same way of building an iteration.

05 Functions in. Functions out.

01:47–03:05 · 144 words · 9s visual holds

Phase 1

> Point to the returned fn, then the captured n.

Let’s improve a guess for a square root. In Clojure, a call puts the function first, then its arguments. Defn names a function; square brackets list parameters. This function takes n and returns an update that averages a guess with n divided by that guess. We use positive inputs, avoiding division by zero. The outer function chooses the problem; the returned function performs one update.

Phase 2

> Right. Read the three independent results.

Here, map applies that update separately to three guesses. Returning this function makes the constructor higher-order. Map is higher-order because it takes the returned function as an argument. Each input gets one update. There is no connection between the three results. In Clojure, fn introduces an anonymous function, just as a lambda would elsewhere.

Phase 3

> Right. Point from one result back to the input.

But to solve the problem, I want the result of one update to become the input to the next. That is what iterate gives me.

06 iterate: state → state → …

03:05–04:07 · 98 words · 15s visual holds

Phase 1

> Point to state 0.

The first element of iterate is the starting value itself: one. No update has happened yet.

Phase 2

> Right once.

One update gives one point five. We keep it as the next state.

Phase 3

> Right once.

Now the function receives one point five, rather than the original one. The new guess is about one point four one seven.

Phase 4

> Right once.

The next guess is closer again. Each call has exactly the same interface: a guess in, a guess out.

Phase 5

> Right once. Pause on the complete chain.

Take asks for five states from this lazy sequence. Iterate itself has no idea what a square root is, or when we should stop. We supply that knowledge.

07 A fixed point stays the same

04:07–05:06 · 110 words · 6s visual holds

Phase 1

> Follow one trajectory from the starting guess of three.

Let’s find the positive square root of four. This function averages our guess with four divided by that guess. If the guess is too high, that second number is too low. Starting at three, repeated updates give about two point one six seven, then two point zero zero six four, then almost two. These are successive calls, each using the previous result.

Phase 2

> Right. Explain why the solution is also a fixed point.

At two, the same calculation returns two. More generally, a positive number left unchanged by this update must satisfy x squared equals four. That is why finding its fixed point solves our equation. The connection depends on the step we wrote. Iterate itself promises neither improvement nor convergence.

08 step + converged?

05:06–06:22 · 140 words · 9s visual holds

Phase 1

> Point to the parameters and iterate, not every implementation line.

Now we separate two jobs. Step tells us how to update a state. Converged tells us whether two successive states are close enough. The question mark is part of the predicate's name. This helper returns the trace, so we can inspect it. The function and stopping test are both arguments.

Phase 2–penultimate

> Advance through the guesses; do not read all decimal places. This paragraph covers every intermediate numerical phase; do not narrate each number.

For the square root of two, I use a tolerance. Watch the values settle. The maximum number of updates is a cap on work, not a request to compute every one. The lazy sequence is consumed only until the stopping test succeeds or that cap is reached.

Phase last

> End if needed. Pause on the stopping message.

The change is below ten to the minus twelve. Successive change does not automatically bound solution error; hitting the work cap is also different from convergence. Here is the full helper for reference. Its important interface remains the supplied step and stopping test.

09 Plan tomorrow’s turbine settings

06:22–06:47 · 46 words · 3s visual holds

Phase 1

> Point to the forecast and the turbine scene.

Our first problem is a wind farm. Six turbines, twelve hourly decisions each: seventy-two settings to choose. Wind sets the available electricity; we choose how much to retain. At noon, this example offers eighty-four kilowatt-hours per turbine. Setting one retains it all; one half retains forty-two.

10 What makes one plan better?

06:47–08:18 · 171 words · 9s visual holds

Phase 1

> Read the three whole-plan rows.

If electricity were our only concern, we would retain everything. Stopping turbines gives up electricity. It also reduces the exposure of birds passing the farm. We need to say how we compare those two goals. The cost function on the left adds up electricity lost across the whole plan. Clojure maps can be called as lookup functions: run applied to a turbine-hour key returns that setting. For produces the contributions; reduce plus adds them.

Phase 2

> Right. Point to T2 and T3 at noon.

The turbines are not equally exposed. Each cell here is exposure at full operation for a turbine and an hour. T3 is close to the flight path; T2 is farther away. The hourly traffic pattern is shared, and location scales it. So curtailing T3 at noon removes much more exposure for the same electricity loss. This is an illustrative exposure index, not a prediction of birds killed.

Phase 3

> Right. Return to the tradeoff table.

Neither column alone tells us which compromise to choose. First I'll show how we improve a plan for one numerical cost. Then we'll supply a cost that includes both concerns.

11 Optimization is iterate

08:18–10:00 · 194 words · 9s visual holds

Phase 1

> Point to the slope on the graph, then the returned fn.

Suppose we score only electricity lost. The graph shows how that cost changes when one setting changes. A downhill direction reduces the cost. Gradient takes our cost function and returns a function that estimates the slopes at a supplied plan. It does that numerically, without symbolic differentiation.

Phase 2

> Right. Read one row, then gesture across all six.

Descent-step subtracts a small multiple of each slope. Half minus point zero zero zero one times minus eighty-four gives point five zero eight four. The argument destructuring extracts x from the state map. Assoc returns the updated map without changing the old one. Hash-parenthesis is a shorter way to write a lambda; percent denotes its argument. All seventy-two coordinates use the same old vector. The noon settings move equally here because their electricity forecasts are equal.

Phase 3

> Right. Follow solver-step from the inside outward.

Clamped takes that update function and returns another function. It calls the update, then clips every proposed setting to zero through one. The solver is now a function from a plan state to a new plan state. Iterate feeds each new state back. All the problem-specific work is inside the step we supplied. Before we change the cost, let’s look at the small multiplier that controls how far each update moves.

12 How far should one update move?

10:00–10:30 · 51 words · 6s visual holds

Phase 1

> Return to the electricity-only slope at turbine 1, noon.

The rate we just supplied controls the movement. Keep turbine one at noon, with slope minus eighty-four. At this rate, half becomes point five zero eight four.

Phase 2

> Right. Point to the changed rate and result.

Double the rate and this proposed movement doubles. For this electricity-only cost, running everything is obviously best. Now let’s supply the tradeoff with birds.

13 Which plan should win?

10:30–12:20 · 209 words · 9s visual holds

Phase 1

> Point to A and B, then the electricity row.

Now let's choose the tradeoff. Plan A retains half everywhere. Plan B is identical except that T3 stops at noon. B gives up another forty-two kilowatt-hours. On electricity alone, A wins.

Phase 2

> Right. Explain the small risk-14 example before the table.

I choose a target of twelve for total exposure. Above that, I add one hundred times the squared excess to the cost. A risk of fourteen is two above target, so it adds four hundred. Twelve and one hundred are modelling choices. The weight converts squared exposure into score units comparable with the electricity-loss term.

Now B wins: its reduction in exposure is worth more under this chosen score than the extra electricity lost. Both candidate plans still exceed the target; we have compared a direction of improvement, not finished the optimization. This is a soft penalty. It encourages lower exposure; it does not guarantee a strict limit.

Phase 3

> Right. Point to cost, risk and the returned fn.

Here is the wrapper. It receives the original cost, a risk function, and the parameters, then returns a new cost function. Comp in the definition composes functions right to left: first turn the vector into a plan, then measure its risk. Both old and new costs accept the same vector and return one number. Gradient can sample either one. We change the objective without changing the update algorithm or iterate.

14 Same updates, different objectives

12:20–13:48 · 150 words · 16s visual holds

Phase 1

> Explain the grid before pressing Space.

Both sides start with the same plan, rate, and stopping test. A row is one turbine; a column is an hour; darker means more generation retained. The only difference is the supplied cost function.

Phase 2–penultimate

> Space to play; pause near the middle if useful. Speak over the changing grids. This paragraph covers every intermediate numerical phase; do not narrate each number.

On the left, electricity alone pushes the settings upward. On the right, the bird penalty changes which settings are worth increasing or reducing. The plan changes most where traffic and exposure are high. These frames are successive drafts of tomorrow's plan. The day itself is not passing yet.

Phase last

> End. Read the displayed final generation and risk, without memorizing them.

Compare the final generation and exposure. The penalty buys a reduction in exposure by giving up some generation. Here both traces stop because the largest setting change is below point zero zero one. The cap is six hundred updates. That says the numerical search has settled under our tolerance; it is not a general proof of optimality. Could a different update reach a settled plan in fewer steps?

15 Can momentum reduce the number of updates?

13:48–14:34 · 83 words · 6s visual holds

Phase 1

> Point to the returned fn and the velocity in state.

We have just watched plain gradient descent on the bird-aware cost. Here is a different update for that same problem: momentum. It calls the base step and adds ninety percent of the previous movement. That velocity travels in the immutable state. Iterate still receives a state-to-state function.

Phase 2

> Right. Follow the wrappers inside out; read the measured table.

The base step performs descent; momentum wraps it; clamped wraps the result. Clamping clips position but keeps velocity. We have changed the solver's update without changing the model. Now let's replay the plain solver's resulting day.

16 Play the resulting day

14:34–15:30 · 89 words · 14s visual holds

Phase 1

> Watch the highlighted bird reach the rotor, flash red and fall.

Now we play a day through a fixed schedule. The red rectangle marks the current hour. First, every turbine runs. Watch the highlighted bird hit a moving rotor and fall. This is an illustrative strike; our cost model measures exposure, not fatalities.

Phase 2

> Right. Watch the same crossing during busy hours when the exposed turbine is stopped.

With the penalty, the chosen schedule stops this exposed turbine during busy hours. The same illustrated bird now passes through. When the turbine runs again, the strike can still appear. The clock is advancing; the plan is no longer changing. The optimizer produced the schedule being replayed.

17 Buy hydrogen before the customers arrive

15:30–17:36 · 236 words · 12s visual holds

Phase 1

> Point to tank capacity, supplier table, then customer probabilities.

For decision-making, consider a hydrogen station. It has room for four units, and each customer truck takes one. The station buys from a supplier before it knows how many customers will arrive. Regular weeks offer up to three units at a price of two. Tight weeks offer only one unit, at a price of six. Those weeks alternate. The tank's physical capacity stays four; the supplier's available quantity changes.

Phase 2

> Right. Follow the delivery into the tank.

Customer arrivals are uncertain: forty percent of weeks have no trucks, fifteen percent have one, and forty-five percent have a convoy of three. The average is one and a half, but the full distribution matters. We use the same distribution each week, independently.

Here we buy one unit in a regular week, bringing our stock to two. The order is chosen before we see which customer outcome happens.

Phase 3

> Right. Point to the turned-away truck.

A convoy arrives. We sell to two trucks and turn the third away. This is one possible outcome, not the demand we knew when choosing the order.

Phase 4

> Right. Point first to reward, then next-state.

We sell each unit for ten, so this outcome earns twenty minus the purchase cost of two: eighteen. Unsold units would cost one each to hold for the week. The next state contains leftover stock and the next supply window. Our objective is expected profit over repeated weeks, multiplying each following week's contribution by another factor of point nine. This is a modelling preference for earlier returns, not a ten-percent hydrogen leak.

18 Buy for this week—or the next?

17:36–18:08 · 61 words · 3s visual holds

Phase 1

> Read the state and compare the two order columns.

The state is a pair: supply week and stock. Orders returns feasible purchase quantities.

In a regular week with one unit already in stock, buying two covers the largest possible convoy this week. Buying three fills the tank to four. Why carry more than any current convoy needs? Look at the leftovers: either way, the next week has expensive, limited supply.

19 Future values change the order

18:08–21:35 · 400 words · 15s visual holds

Phase 1

> Keep the state fixed throughout all four reveals.

V is a table: for each supply week and stock level, it estimates profit from then onward. Start all entries at zero. Clojure maps are functions too, so V of a state is a lookup in that table. For each possible order, average over the three demand outcomes: this week's profit plus point nine times the value of the next state. With every future value zero, only this week matters. Order two wins.

Phase 2

> Right. Point to the unchanged best order.

After one update, the table contains optimal one-week values. Using those as the future term now scores a current order plus one more week. We score the same orders again, from the same initial situation. Order two still looks best. Nothing has been bought; this is calculation.

Phase 3

> Right. Identify the jump to update three.

We skip update two on this slide. After three updates, V accounts for three weeks; scoring an order with it looks one week further ahead. Now order three wins. The extra stock has value because it carries us into the tight supply week. We did not add an if statement saying to buy more before a tight week. The recommendation follows from the transitions, costs, and future values.

Phase 4

> Identify q as expected-return. Read the order scores, then distinguish the three-week and four-week horizons.

Here, q is the model's expected-return function. Give it V, a state and a proposed order. It averages this week's profit plus the discounted value of next week's leftover stock over the possible demands.

For regular week, stock one, the four orders score 19.10, 23.64, 28.04 and 28.35. Max keeps 28.35. Bellman repeats that calculation for all ten states—two supply weeks times five stock levels—and returns a new value table.

The old entry was 23.06. It covered three weeks from this state. The new entry covers four: this week plus the three-week values of possible next states. That increase is not a measured saving from changing the order. No hydrogen has been bought; we are extending the planning horizon.

Phase 5

> Keep the same V. Contrast the winning score, 28.35, with the winning purchase quantity, 3.

Now we use the same input V and the same scores, but keep the winning order instead. Max-key returns three because ordering three scored 28.35. Three means units to buy, not profit or target stock. We already hold one unit, so buying three fills the tank to four.

Best-order does this for every state and returns a map from state to order—a policy. Clojure maps support function calls, so the expression below looks up this state and returns three. It neither updates V nor uses the new value table from the preceding reveal.

20 Hydrogen: iterate the value table

21:35–23:09 · 166 words · 14s visual holds

Phase 1

> Identify the axes and colours before starting the animation.

Now iterate updates the whole table of profit estimates. Each pair of bars is a starting stock level: blue for a regular week, amber for a tight week. Height is money, not hydrogen or an order. The separate best-order call produces the order column. Both use the same model.

Phase 2–penultimate

> Space. Follow the stock-1 row; allow the animation to finish. This paragraph covers every intermediate numerical phase; do not narrate each number.

Every update revises all ten values using the old table. Best-order then compares possible purchases using those values. Watch stock one: the recommendation changes as the estimates account for more future consequences. No purchases happen during this animation; it is planning.

Phase last

> End. Point to the residual and the two stock-1 recommendations.

The largest change is now below our tolerance. In the stopping lambda, percent-one and percent-two stand for the two successive tables. With one unit in stock, the recommended action depends on the week: buy three when supply is cheap, and buy none in the tight week. For a fixed finite model and discount below one, exact value iteration has a convergence argument. Later I'll show the bound behind it; here we are watching a floating-point calculation.

21 The best order depends on the supply window

23:09–24:29 · 154 words · 6s visual holds

Phase 1

> Read stock 1 first; compare the strongest simple rule with Bellman.

The policy maps supply window and stock to a purchase quantity. “This week only” maximizes immediate expected profit, with no value assigned to leftover stock. Bellman also accounts for future weeks. We evaluate every rule over the same twenty discounted weeks, weighting all demand branches. Filling toward three is already good; Bellman does a little better. This is a comparison of stationary policies, not an optimization for a business closing after week twenty.

Phase 2

> Right. Point to the empty-tank tight-week row.

There is a less obvious recommendation here. With an empty tank in a tight week, buy one, even though that purchase has negative expected profit this week. The cost is paid now. If nobody arrives, the unit carries into next week, when cheap supply is capped. That future benefit outweighs the immediate expected loss in this model. Looking only at immediate expected profit misses that value. This comes from the actual model; it is not a general claim about hydrogen markets.

22 Validate the turbine controller

24:29–26:35 · 239 words · 12s visual holds

Phase 1

> Return explicitly to the wind farm; read input, controller, output.

Back at the wind farm, we ask a different question. A confident bird detection must cause a STOP command in that modelled minute. Can any event sequence leave the command at RUN instead? Detection is an input. We are validating the controller's response, not a bird-recognition model.

Phase 2

> Right. Point to tick and detect branches.

Here is one controller. A timer makes a decision every ten minutes. A detection arriving between ticks is recorded, but the action is unchanged. That can leave the turbine commanded to run after a confident detection. We have a concrete rule and a concrete failure.

Phase 3

> Right. Explain the missing table row before reading the wrapper.

We can wrap the controller so a stronger detection triggers an immediate decision. But its decision table has another problem: an unfamiliar situation has no row. The default stopping probability is zero. The variable u is a sampled draw compared with that probability. Our finite exploration represents the relevant draw bands explicitly, rather than sampling during the search. Reacting sooner does not fix a bad decision. In this table, a confident detection in a known situation does stop the turbine.

Phase 4

> Right. Follow with-shield and compare the two computed outputs.

With-shield wraps the decision function. In known situations it delegates to that function. In an unknown situation, any detection produces STOP. The same event now gives STOP instead of RUN. The construction below shows the wiring: the shielded decision is supplied both to the timer controller and to the immediate-reaction wrapper. One example is encouraging. Next we search all the event sequences represented by this finite model.

23 Can any event sequence reach RUN + confident bird?

26:35–29:11 · 269 words · 27s visual holds

Phase 1

> Point to the rule predicate and the initial state count.

Our search state is now a set of controller states. The predicate at the bottom says what failure means: detection level two and action RUN. Keywords such as det and action select fields from a map. We start from the controller's possible initial states.

Phase 2

> Right. Explain the branch diagram and the set-expansion code together.

The next minute can contain no detection, a weak one, or a confident one. The diagram shows one branch; the counts cover the full exploration. Reach-step takes that successor function and returns a set update. Mapcat collects successors; into adds them to the states already known. A set removes duplicates. Detection and timer transitions are separate so a reset cannot hide a failure.

Phase 3

> Right. Pause on the timer-only failure count.

Repeat until adding successors changes nothing. For the timer-only controller, states violating the rule are reachable. The search has found a real problem in this model.

Phase 4

> Right. Identify the change of controller.

Now swap in the controller that reacts immediately. Keep the same search, event model, and failure predicate.

Phase 5

> Right. Point to the unseen-situation branch.

The unknown-context branch still leads to RUN after a confident detection. That missing table row survives the timing fix.

Phase 6

> Right. Pause on the second failure verdict.

The second controller still fails. We need both the immediate reaction and a safe decision for the unknown situation.

Phase 7

> Right. Identify the shielded controller.

Finally, supply the shielded controller to the same exploration.

Phase 8

> Right. Point to STOP on confident detection.

The branch that failed before now issues STOP.

The search continues, so we are checking more than that one branch.

Phase 9

> Right. Pause on set closure.

The set is now closed under every encoded successor, without a failure. That establishes the rule for this finite model and its initial states. Set equality is the stopping test. Finiteness and the growing set guarantee eventual closure; infinite-state systems need not terminate. This says nothing about sensor accuracy or physical braking.

24 iterate underneath all three

29:11–29:59 · 92 words · 3s visual holds

Phase 1

> Read across each row; do not repeat the whole story.

Underneath all three examples is iterate. What changes is what a state means. A vector of turbine settings becomes a value table, then a set of reachable states. The step changes with it, and so does the stopping test. Iterate exposes the intermediate states as a trace we can inspect. This is an implementation pattern, not a claim that the three algorithms have the same guarantees. Set closure finishes a finite search; small numerical movement needs a separate interpretation. We already used that interface to change the optimizer’s pace and add momentum.

25 Rainbow

29:59–30:20 · 41 words · 2s visual holds

Phase 1

> Introduce Rainbow's focus; then distinguish it from Lean.

Rainbow is a theorem prover and code generator focused on Clojure, operations research and related fields. Its purpose is not to match or beat Lean. I want to show you what that focus means, and a few things we have proved.

26 How Rainbow differs from Lean

30:20–30:59 · 74 words · 3s visual holds

Phase 1

> Compare focus and workflow; acknowledge the shared capabilities.

Lean is a general-purpose theorem prover and programming language. It already has proof objects, metaprogramming and native compilation. Rainbow focuses on Clojure and operations research. Its explicit proof terms are EDN values we can manipulate with Clojure functions. Its code generator works with a modeled subset of Clojure. Both systems check typed proof terms with a kernel. I am choosing a particular workflow and application area, not claiming that Lean cannot do this work.

27 A proof we can edit

30:59–31:59 · 107 words · 9s visual holds

Phase 1

> Read the composition, then match the app nodes to the term.

Here is a small proof in Rainbow syntax. Given f from P to Q, g from Q to R, and evidence p of P, apply f then g to produce evidence of R. This is function composition. The tree is drawn from the proof term itself.

Phase 2

> Right. The outer application disappears from both the term and its tree.

Remove g and the term produces Q, while the claim requires R. The kernel rejects it. A well-formed piece of data is not necessarily a valid proof.

Phase 3

> Right. Point to the function passed to update-in; g returns to the tree.

Use update-in to insert g again. The resulting term checks. These are recorded kernel results; the browser edits and draws the data. This is how ordinary Clojure functions can help us work with proofs.

28 What we have proved

31:59–33:06 · 127 words · 6s visual holds

Phase 1

> Read the action scores, then widen the claim from this example to every allowed input.

One result says that selecting the best action from a finite set really returns a score at least as high as every candidate. B wins these three scores. The theorem covers every allowed finite set and exact nonnegative rational score function. That is a property of the mathematical selector, not a proof of our browser code.

Phase 2

> Explain the upper bound before interpreting the matching numbers.

Another result is LP weak duality. Given a feasible plan and a valid dual certificate, the certificate bounds the plan’s objective. Suppose a plan earns ten and a certificate says no feasible plan can earn more than ten. Then that plan is optimal for the model. Rainbow has checked the general theorem under its exact nonnegative-rational assumptions. The plant checker still needs the connection from this mathematics to its own program.

29 From a specification to Clojure

33:06–33:47 · 79 words · 3s visual holds

Phase 1

> Recall the turbine bounds. Read the generated function and three example inputs.

This is Clojure generated by Rainbow for a clamp function. We used the same operation to keep turbine settings between zero and one. Minus one becomes zero; one half stays one half; two becomes one. These are recorded executions of this generated function. Separately, Rainbow checked that the modeled program implements clamp, and that compilation preserves the meaning of properly scoped programs. The printer, exact-arithmetic primitives and runtime remain trusted. This is a small generated program with specific guarantees.

30 Scheduling an e-methanol plant

33:47–34:20 · 62 words · 3s visual holds

Phase 1

> Explain the scheduling problem, then state the measured benchmark outcome.

I built this illustrative methanol model to meet a weekly production contract while choosing when to buy electricity and use the hydrogen tank. For all fifty-two tested weeks of 2023, we obtained a schedule and an accepted certificate. The largest reported cost gap was twenty-six cents. The benchmark knows the whole week’s prices, and the checker’s general soundness proof is still open.

31 52 weekly certificates accepted

34:20–34:54 · 66 words · 3s visual holds

Phase 1

> Attribute each contribution, then give the measured checking time.

HiGHS finds the proposed schedule from our Clojure model. Our branch-and-bound algorithm builds the certificate tree, choosing branches and constructing exact bounds with HiGHS solving the smaller problems. Rainbow checks that evidence with exact arithmetic. All fifty-two weekly certificates were accepted; the median check took point six four seconds per week. That is checking time, not the time to find a schedule or build its certificate.

32 San Roque: what can scheduling save?

34:54–35:29 · 65 words · 3s visual holds

Phase 1

> Say this is a separate e-SAF model, a baseline comparison and a modeled result.

For financial scale, take our separate e-SAF model at San Roque, by the Bay of Algeciras, using local wind and solar data. Planning forty-eight hours ahead saves a modeled average of 4.4 million euros annually in energy and backup hydrogen against the best tuned operating rule. This excludes missed-delivery penalties. These are model estimates, not measured factory savings or results covered by the methanol certificate.

33 Where I want to take Rainbow

35:29–36:00 · 59 words · 3s visual holds

Phase 1

> Return to proofs as data. Describe a direction, not a finished editor.

I want an operator to open a certificate and ask: which constraint ruled out this schedule? Which assumption supports this bound? We could show the relevant part, insert a step, then check the result. Text need not be the only interface. Our tiny example does that for composition. The broader OR views and editing tools are still to build.

34 Applied Higher-Order Functions

36:00–36:25 · 47 words · 3s visual holds

Phase 1

> Gesture to the book photograph; leave the URL visible.

I wrote Applied Higher-Order Functions about using higher-order functions in operations research. But the ideas apply more broadly, especially if you design algorithms that optimize or help make decisions. The book has longer explanations and runnable examples. You can find it at flexiana dot com slash book.

35 Thank you

36:25–36:37 · 18 words · 3s visual holds

Phase 1

> Leave the contact details visible for questions.

Thank you. You can reach me at jiri at flexiana dot com. I’d be happy to take questions.